Introducing Context-Fabric:

Scalable Corpus Analysis for the Agentic Era

Cody Kingham

Aug 2026

Abstract

For ten years, Text-Fabric (TF) has served as a powerful tool for corpus analysis, underlying an entire generation of linguistic and exegetical research. However, TF has always been burdened by two problems: 1) it is resource intensive, consuming large amounts of RAM; and 2) it requires knowledge of Python or a query language to use it. These frictions have limited its usefulness to the philological community, which is not known for its computational prowess! Agentic AI, however, now makes it possible to solve the skill-gap. To maximize this opportunity, TF must become light and agile, capable of running on resource-constrained systems, with the ability to scale to many corpora at once. To address this, I introduce Context-Fabric (CF), a memory-efficient re-write of TF designed for the agentic era. Using memory-mapped arrays to store the corpus graph, CF achieves up to a 92% reduction in RAM consumption, with as much as 5.3× more efficient scaling for multi-corpus loads. This is accomplished without sacrificing search speeds, which are 3% *faster* overall, with direct feature look up improving 26%. To take advantage of this power, CF comes with an MCP server optimized for AI agents. This makes it possible to conduct complex, high-level research without knowing a jot or tittle of Python.

1 The Legacy of Text-Fabric

In 1977, Eep Talstra founded the Werkgroep Informatica at the Vrije Universiteit Amsterdam (today the Eep Talstra Centre for Bible and Computer, or "ETCBC") with the purpose of using computers to analyze the Hebrew Bible. Over the next fifty years, those efforts yielded a rich grammatical database of Biblical Hebrew. For most of this period, the dataset was stored privately on the Werkgroep's internal server.[1] It was not until 2014 that Dirk Roorda, a research scientist on the SynVar project (*Does Syntactic Variation Reflect Language Change? Tracing Syntactic Diversity in Biblical Hebrew Texts*), published the dataset online as an XML dump in Linguistic Annotation Framework (LAF) [2]. LAF models texts as graphs. The various linguistic units like clauses and phrases can be captured as nodes with features, and their relations as edges [3]. Alongside the XML dump, Roorda published a Python package called LAF-Fabric, which could walk the nodes and edges to query their features [2]. By choosing Python, Roorda simultaneously introduced Biblical Hebrew to the burgeoning fields of Data Science and Machine Learning, which had begun to widely adopt Python as the default scripting language. One could, within the same program, collect statistics on a given grammatical pattern and then apply complex statistical algorithms with tools like NumPy, Pandas, Scikit Learn, and others [4, 5, 6]. These new capabilities were soon picked up by enthusiastic researchers who leveraged them for a whole host of studies. And they brought new emphasis on data-driven, reproducible results [8, 7, 9, 12, 10, 11].

In 2016, Roorda improved on the idea by releasing Text-Fabric (TF) [13]. TF dramatically reduced the disk storage needed for the graph data by replacing in-line XML with a stand-off

[1] It was also integrated into several paid software tools such as Quest and its successor, the Stuttgart Electronic Study Bible (SESB) [1].

plain-text format, the `.tf` file. It made the graph data modular so that each `.tf` file contributed a single node feature or edge relationship. If one only wanted to analyze verb tense, for instance, he could choose to load only the `vt` feature.[2] The `.tf` format is a single column of data, with each row indexed by its node identifier. This strips away the syntactic overhead required by the XML format, reducing disk space and load times. Roorda envisioned that this new nimble format, along with a host of user-friendly visualization tools, would catalyze a new category of scholar, the "programming theologian".

The vision of the computational theologian was certainly ambitious, and it dovetailed well with the ETCBC's heritage; but its success was always hampered by one minor problem: learning to code is hard. And that difficulty was compounded by the right-brained tendencies of humanities students who were more comfortable writing exegetical summaries than loops and conditions. That is not to say the idea failed. Indeed, the study of syntax is closely aligned to left-brained tendencies, and many enterprising scholars followed the Roordian path. But the dream's true fulfillment—of a humanities field radically transformed by a *scientific* approach—simply could not be realized given the skill gap.

Such was the state of things until 2022, when OpenAI announced on Twitter the release of a new public demo of its Large Language Model (LLM), ChatGPT [14]. This is the "AD" moment that will, most likely, come to separate the eras of manually-written and machine-generated code. The first beachhead was autocomplete, which could generate entire functions, fit for purpose, on the fly. Yet, most code continued to be written by hand within closely-supervised IDEs. For many engineers, the true moment of surrender came around December 2025, when Anthropic released its `Opus 4.5` model alongside its *Claude Code* tool. Here was a tool which could reliably generate entire software packages in one go, complete with tests and documentation. Since then, the trend towards agentic-authored code has sharply accelerated with the encouragement of additional models and tools (e.g. Codex, Grok Build, open source weights) and competitive pressure.

The AI revolution now makes it possible to fully realize Roorda's vision, closing the skill gap that made Text-Fabric inaccessible for most. To maximize this opportunity, several transformations to Text-Fabric are required. First, corpus search must become *deployable* to a production environment. This means the ability to serve search from a persistent web server for thousands of requests at a time. This will enable AI agents to search remotely from any sandbox environment they happen to be running in. This requires a change to the way TF accesses the text graph. The current approach loads all nodes and edges into RAM, consuming a significant amount of memory. In production, especially under high loads, this is untenable. Memory mapping offers an alternative where graph data can be stored on disk and accessed only when needed. This method is already used by a number of performance-sensitive machine learning tools like PyTorch and TensorFlow [15, 16]. Second, Text-Fabric must be enhanced to automatically build and return corpora descriptions that can be consumed by AI agents. These descriptions are crucial for agents to know what nodes, edges, and features are available for searching. Ideally, these descriptions would be completely corpus-agnostic so that they work out-of-the-box with pre-existing corpora. Additionally, Text-Fabric should return a syntax guide for running TF queries. The guide should be optimized for progressive discovery, so that agents can drill down to the specific patterns they require while ignoring the rest.

These requirements reach deep into Text-Fabric's data model and radically transform what is possible. For this reason, and in the spirit of TF's previous evolution from LAF-Fabric, I have chosen to rename and re-release this tool as Context-Fabric (CF). The name is a nod to the critical role of context engineering in agentic applications. The package is released under the same MIT license under a new shared organization, open for pull requests. CF implements a memory mapped data model using NumPy's `memmap` function with `.npy` files, replacing TF's in-memory Python dictionaries. Several optimizations are applied to ensure that the latency

[2] Plus some required node type and slot edge relations.

penalty of disk streaming is compensated for. In the end, CF pulls slightly ahead of TF for search speeds. Meanwhile, RAM usage drops sharply, and load times go from several seconds to nearly instantaneous. For corpus description and search syntax, CF provides a set of API methods which automatically analyze and sample the text graph, giving the agent a full map along with illustrative examples. This works even with various transcription formats. The search guide is implemented with different "menus", allowing the agent to drill down to specific sub-topics as needed. These methods are wired to an optional MCP server (Model Context Protocol) which ships alongside CF.

2 Migrating Text-Fabric's Data Model

2.1 The Text as Graph

Text-Fabric's data model is the "text as graph". A corpus is conceived of as a collection of nodes (objects of interest), which may have edges (relationships). Different types of attributes, or features, can be stored on nodes and edges. Figure 1 illustrates the organization. All nodes in the graph have an object type feature, `otype`. The only obligatory `otype` is the slot, which defines the atomic units (e.g. words, letters) which comprise the text. There is a corresponding, obligatory edge value from slots to other nodes, which is called `oslots`. This edge defines the containment relationship between a given node and its slots. For example, node `503201` in Figure 1 *contains* slots `1` and `2`, and therefore has an `oslots` edge extending to both nodes. Containment between non-slot nodes (e.g. phrase in clause) is automatically inferred from slot intersection, and need not be encoded separately.[3]

Descriptive attributes, or features, can be stored on both nodes and edges. For example, the first phrase node can have a `typ` feature with a value of `PP`, for "prepositional phrase type". Likewise, embedded clauses may have an edge to another clause with the name of `rela` and a value of `Attr` for "attributive clause". Text-Fabric itself is agnostic about which kinds of nodes and edges a corpus encodes, with the exception of the aforementioned obligatory types. The text-as-graph model is powerful in its flexibility and expressiveness. For this reason, it is desirable to retain it.

2.2 Text-Fabric's Python Dictionaries

In Python, dictionaries are container classes that provide interactions with a hash table. In plain terms, they efficiently route a given key to a given value, and are used for storing data as key-value pairs. Thus, given a dictionary named `oslots`, one can use a node number to very quickly look up the corresponding slot values. This architecture underlies Text-Fabric's corpus loading algorithm, which populates a collection of dictionaries into memory. The approach has the advantage of being very fast, with the tradeoff of massive memory consumption. Large corpora can consume gigabytes. The ETCBC's `.tf` Hebrew Bible, for example, takes approximately 6.3 GB of RAM when fully unfolded. The in-memory dictionary based storage is the critical component that needs to be changed to make loading and usage efficient.

2.3 Memory Mapped Arrays

An alternative way to store key-value pairs is with an associative array using integer or ordinal encoding. For string-based values, this requires two arrays: one to store all of the unique strings for a given feature, and one to store ids corresponding to a given string value, at a given index position.[4] The index position thus corresponds with a node id:

[3] The arrows in the figure, in this sense, do not directly represent edge relations.

[4] For integer-based values, CF can simply store the integer value directly in the `indices` array.

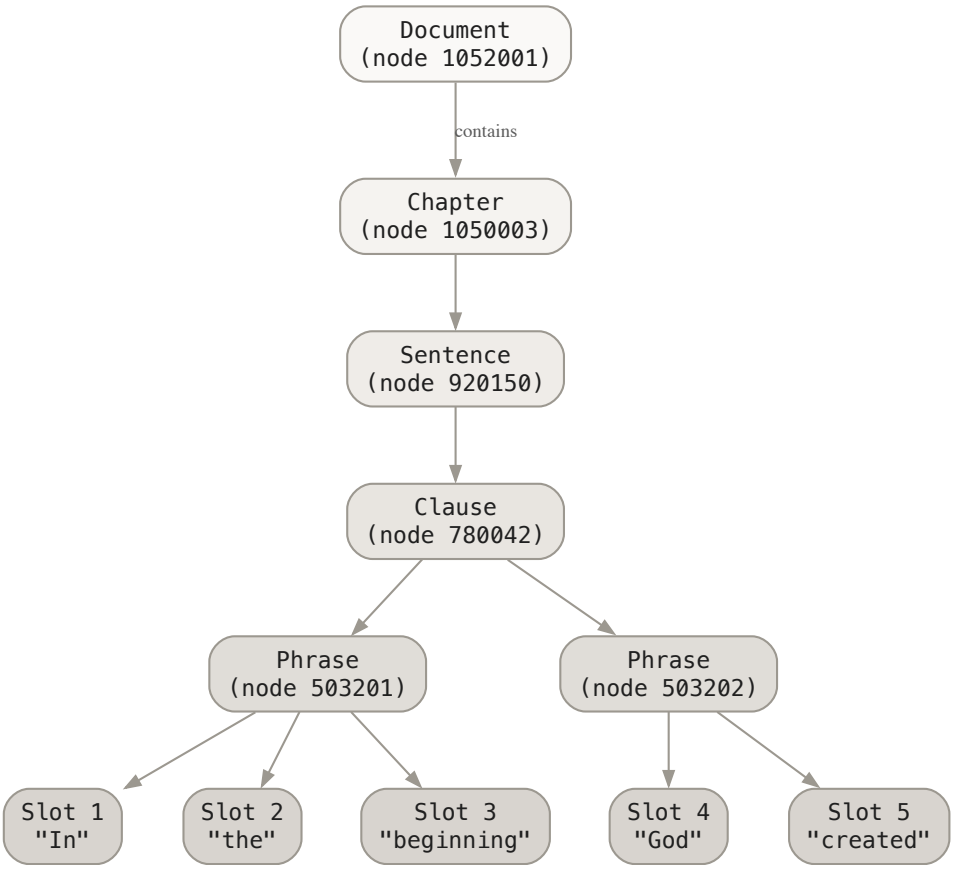


Figure 1: A text-as-graph containment hierarchy. Non-slot containment is inferred from the slots shared by each object.

```python
MISSING = 0xFFFFFFFF  # Sentinel for "no value" (max uint32)
gloss_strings = np.array(["king", "the", "house"], dtype=object)  # Unique
    values
gloss = np.array([0, 1, 0, MISSING, 2], dtype=np.uint32)  # Per-node index

def lookup(node):
    idx = gloss[node]
    return None if idx == MISSING else gloss_strings[idx]

lookup(2)  # -> "king"
lookup(3)  # -> None (missing value)
```

Listing 1: String pool structure

The benefits of using associative arrays are two-fold. First, it allows CF to take advantage of NumPy's `memmap` function for reading memory-mapped files. This process enables a file's contents to be directly mapped into a process's virtual address space [17]. CF stores these arrays as `.npy` files and loads them with `mmap_mode='r'` (read-only) for safe multi-process access. Second, as discussed further below, it allows CF to take advantage of fast, vectorized lookup operations for bulk search, a feature TF lacks.

Because python strings cannot be memory-mapped, CF loads string pools into memory. Since strings are deduplicated, the footprint is negligible. For example, BHSA's `gloss` feature reduces 426,590 words to about 9,000 unique glosses. For nodes which lack the feature in question, a sentinel value is used in place of `NaN`: `0xFFFFFFFF`. This is the maximum `uint32` value, $2^{32} - 1$. A collision with this value would require over four billion unique strings, which is unlikely.

2.3.1 One-to-Many Edge Relations

A unique challenge is posed by edge relations, which can be one-to-many. `oslots` is the most prominent example, and critical for the `.tf` data model. Consider a three-word phrase which contains slots `[5, 6, 7]`. TF stores this data as a tuple value in the dictionary. However, tuples cannot be memory mapped. CF solves this problem using Compressed Sparse Row (CSR) format using a separate index pointer array (`indptr`):

```python
# step 1: Concatenate oslot spans for all nodes
oslots = np.array([
    5, 6, 7,                    # phrase: node 1
    5, 6, 7, 8,                 # clause: node 2
    5, 6, 7, 8, 9, 10, 11, 12   # sentence: node 3
], dtype=np.uint32)

# step 2: record index slice boundaries (exclusive) per node
#     and N+1 final slice boundary
oslots_indptr = np.array([0, 3, 7, 15], dtype=np.uint32)

# step 3: use indptr to look up boundaries
def get_oslots(node):
    start = oslots_indptr[node]
    end = oslots_indptr[node + 1]
    node_oslots = oslots[start:end]
    return node_oslots

phrase_slots = get_oslots(0)  # -> [5, 6, 7]
clause_slots = get_oslots(1)  # -> [5, 6, 7, 8]
verse_slots = get_oslots(2)   # -> [5, 6, 7, 8, 9, 10, 11, 12]
```

Listing 2: CSR format for oslots

For edges with values, CF adds a parallel values array:

```
...
# separate array for edges with values
```

```
values  = np.array(["Adju", "Attr", "Spec"])

def get_edges_with_values(source_node):
    start, end = indptr[source_node], indptr[source_node + 1]
    target_nodes = edges[start:end]
    edge_values = values[start:end]
    return target_nodes, edge_values
```

Listing 3: CSR with values

2.4 File Formats

Context-Fabric uses the exact same `.tf` files as input for corpus data. However, this data must be indexed and saved to disk as a cache to take advantage of memory mapping. This mirrors Text-Fabric's `.tfx` cache, which pickles and saves the optimized dictionaries.

This cache is built when first loading a `.tf` corpus for the first time and is persisted with the `.tf` data in a hidden directory called `.cfm` ("compiled format"). The cache contains the required memory-mapped `.npy` files along with some helper data such as `meta.json` (corpus metadata).

```
corpus/                    # Corpus directory (e.g., bhsa/)
  .cfm/                    # Hidden compiled-format subdirectory
    meta.json              # Corpus metadata
    warp/
      otype.npy            # Node types (uint8)
      oslots_indptr.npy    # CSR boundaries
      oslots_data.npy      # CSR slot data
    computed/
      order.npy            # Traversal order
      rank.npy             # Node ranks
      levup_indptr.npy     # Embedder CSR
      levup_data.npy
    features/
      sp.npy               # Part of speech (dense)
      gloss_idx.npy        # String pool indices
      gloss_strings.npy    # Unique strings
    edges/
      mother_indptr.npy    # Edge CSR
      mother_data.npy
```

Listing 4: .cfm directory structure

2.5 Overcoming Page-Fault Latency

There is an unavoidable tradeoff between memory-efficiency and speed. After all, the whole point of RAM is to provide fast lookup. Memory mapping allows file data to be loaded into RAM when used, and dropped when that RAM is required elsewhere [15]. This prevents large files from blocking swaths of memory. The cost for this behavior is called a page-fault: if a page is not already loaded into memory, it must be read from disk, which is slower than RAM. For performance-sensitive operations like search, this can amount to compounded milliseconds, leading to considerable latency.

To compensate for page faults, CF must improve on TF's search algorithm. As it turns out, the vectorized storage also provides the needed advantage. In TF search, feature matches are found through linear search, i.e. $O(n)$ time complexity:

```
for node in candidate_nodes:
    if feature.value(node) == target_value:
        result.add(node)
```

Listing 5: Per-node iteration (original)

Using vectorized lookup, CF can obtain $O(1)$, significantly decreasing latency for these kinds of operations:

```
node_array = np.array(list(candidate_nodes))
values = feature_data[node_array - 1]
mask = values == target_index
result = set(node_array[mask])
```

Listing 6: Vectorized filtering

The vectorized strategy yields nearly an order of magnitude lower latency (8.2×), translating to 26% faster feature lookups in real-world benchmarks.

Containment search poses a more difficult problem. Because `oslot` containment concerns one-to-many edge relations, the algorithm must perform a random walk, necessarily requiring several lookup operations per node rather than one (as with feature lookup). Because each lookup operation potentially pays the page-fault price, containment search is significantly slower. Text-Fabric already optimizes for containment search by pre-computing containment solutions and storing them in-memory via `levUp` and `levDown` dictionaries. For the largest TF corpus, the BHSA, these dictionaries amount to only `100 MB`. This is a small price for CF to pay in exchange for significantly reduced latency. Therefore, by default CF retains these precomputed values in RAM, offsetting the page-fault penalty.

In summary, with vectorization, combined with a compromise for `levUp` and `levDown` RAM storage, CF compensates for the page-fault penalty, and reaches parity with TF for complex searches. For simple feature-based searches, CF becomes 26% faster due to the $O(1)$ candidate feature lookup gain.

3 Benchmarking and Results

The benchmarks target two main areas: memory consumption, especially under multi-corpora scenarios, and query latency. The tests were conducted with a MacBook Pro with the following specifications:

Table 1: Benchmark environment.

Component	**Specification**
Hardware	Apple M1 Pro, 8 cores, 32 GB RAM, NVMe SSD
OS	macOS Darwin 24.5.0 (arm64)
Python	3.13.11
NumPy	2.4.0
Text-Fabric	13.0.19
Context-Fabric	0.5.0

The tests spanned ten corpora with various ranges of on-disk footprints:
All benchmark code is provided as a modular Python package and is published in the Context-Fabric repository alongside the production code.[5]

3.1 Memory Consumption

There are several approaches to measuring memory consumption [28]. For these tests, I selected Resident Set Size (RSS) to capture the actual physical memory usage of a specific process [29]. Subprocess isolation is used with `multiprocessing.spawn` to prevent cross contamination. `gc.collect()` is called before each measurement:

[5]See `https://github.com/Context-Fabric/context-fabric/tree/master/libs/benchmarks`.

Table 2: Test corpora used in benchmarks, ordered by TF cache size.

Corpus	Description	TF Cache Size
CUC [27]	Copenhagen Ugaritic Corpus	1.6 MB
Tischendorf [25]	Tischendorf 8th Edition Greek NT	34 MB
SyrNT [23]	Syriac New Testament	52 MB
Peshitta [22]	Syriac Old Testament	55 MB
Quran [26]	Quranic Arabic Corpus	73 MB
SP [24]	Samaritan Pentateuch	147 MB
LXX [19]	Septuagint	268 MB
N1904 [20]	Nestle 1904 Greek NT	319 MB
DSS [21]	Dead Sea Scrolls	936 MB
BHSA [18]	Biblia Hebraica (ETCBC)	1.1 GB

```python
def _measure_cache_load(source: str, result_queue):
    """Runs in a clean spawned subprocess."""

    # Load corpus from cache
    fabric = Fabric(locations=source)
    api = fabric.load()

    # Measure total RSS after loading
    gc.collect()
    total_rss = psutil.Process().memory_info().rss
    result_queue.put(total_rss)

# Main process spawns clean subprocess
ctx = multiprocessing.get_context('spawn')
p = ctx.Process(target=_measure_cache_load, args=(source, queue))
p.start()
```

Listing 7: Subprocess isolation pattern

Memory consumption is tested under three different deployment scenarios. Single process measures local use on a researcher's machine or single-threaded batch processing. Spawn mode tests the multi-processing approach most common on Macs and PCs. Fork mode simulates multi-processing in production environments on Linux machines. On both multi-processing tests, the number of workers is set to four.

3.1.1 Results

Table 3 and Figure 2 show memory consumption results for single-process corpus loads for all ten test corpora.[6] Smaller corpora benefit less from CF's memory mapping (20% reduction for CUC). However, large corpora such as BHSA and DSS (Dead Sea Scrolls) show a >90% reduction. On average, the total memory consumption is reduced 65% (±21%).

Load times also show marked improvement, though with more variability. For BHSA, CF shows 12.9× faster load speeds, and 6.2× for DSS. But for smaller corpora the difference is negligible.

Next, I tested multi-corpus scaling effects under a progressive loading scenario, where an additional corpus is loaded into CF / TF at each interval. Corpora are loaded in ascending order of size. The results are presented in Table 4 and Figure 3.

The data reveal supralinear scaling effects in TF, likely due to the compounding memory

[6]The figures are averaged across ten runs with 95% confidence intervals per configuration. The reduction percentages are calculated as $(M_{\text{TF}} - M_{\text{CF}})/M_{\text{TF}} \times 100$.

Table 3: Memory efficiency comparison across 10 corpora in single-process mode. Corpora ordered by TF memory footprint. Load time speedup calculated as TF load time / CF load time.

Corpus	**TF (MB)**	**CF (MB)**	**Reduction**	**Load Speedup**
CUC	164	131	20%	0.52×
Tischendorf	304	146	52%	1.62×
Quran	401	187	54%	1.29×
Peshitta	404	173	57%	0.96×
Syrnt	515	145	72%	3.08×
SP	644	174	73%	4.25×
LXX	1,395	388	72%	1.25×
N1904	1,566	411	74%	2.41×
DSS	3,233	337	90%	6.24×
BHSA	6,292	524	92%	12.91×
Mean	1,492	262	65%	3.45×

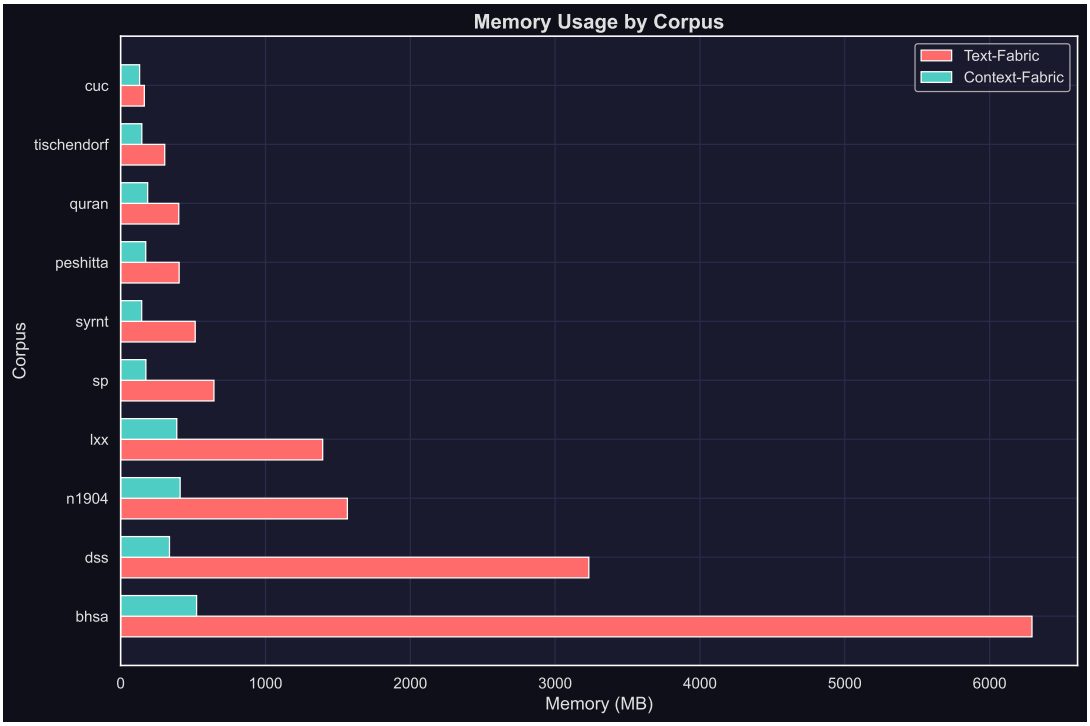


Figure 2: Memory consumption comparison across 10 corpora, sorted by Text-Fabric memory footprint. Error bars show 95% confidence intervals over 10 measurement runs.

Table 4: Memory consumption during progressive multi-corpus loading. Values represent mean total RSS (MB) across 5 runs after loading the specified number of corpora.

Corpora Loaded	TF (MB)	CF (MB)	Reduction	Ratio
1 (CUC)	163	130	20%	1.3×
2 (+Tischendorf)	306	146	52%	2.1×
3 (+Syrnt)	641	161	75%	4.0×
4 (+Peshitta)	866	204	76%	4.2×
5 (+Quran)	1,070	259	76%	4.1×
6 (+SP)	1,531	300	80%	5.1×
7 (+LXX)	2,717	551	80%	4.9×
8 (+N1904)	4,044	791	80%	5.1×
9 (+DSS)	6,088	968	84%	6.3×
10 (+BHSA)	5,529	1,348	76%	4.1×

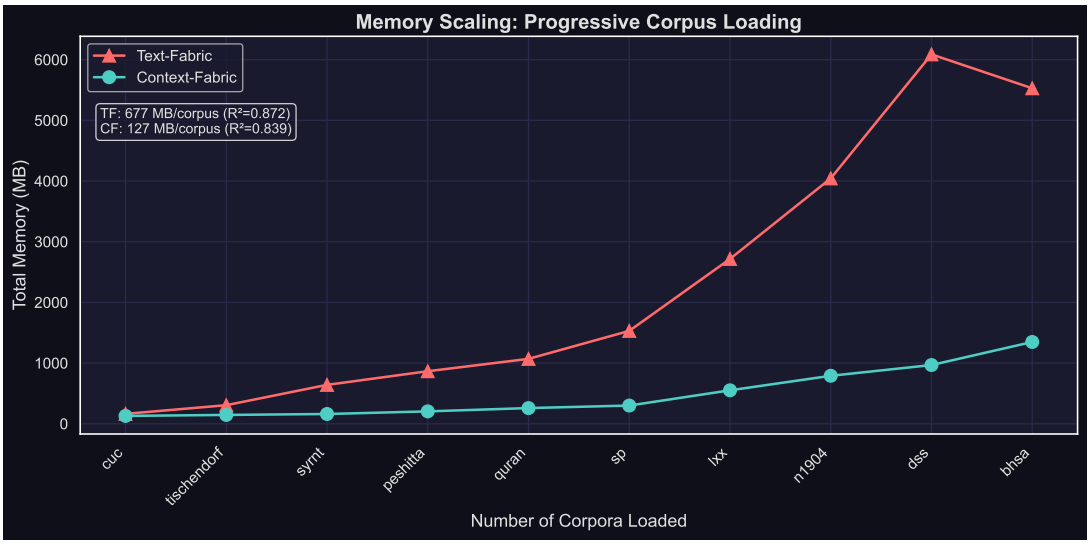


Figure 3: Memory scaling during progressive corpus loading. Quadratic fits show TF grows as $126n^2$ MB (significant compounding overhead) vs. CF at $19n^2$ (6.5× smaller quadratic coefficient).

overhead of Python's object storage.[7] Technically speaking, both TF and CF's growth fit a quadratic line better than a linear one: TF grows as $126n^2$ MB while CF grows at only $19n^2$. The gap, however, is significant, with TF's rate being 6.5× faster. The model predicts that at 50 corpora, TF would hold ∼32,443 MB in memory versus only ∼6,137 MB for CF. For production scenarios where hundreds of corpora might be required, these compounding effects are untenable. CF therefore considerably improves over TF on memory consumption, making this usecase possible.

3.2 Query Latency

Query latency is measured using 100 curated search templates, stratified into four different types, and executed against the BHSA:

- **Feature-based queries** (30): Filtering based on a single feature (e.g., find all verbs, filter by part of speech)
- **Structural queries** (30): Pattern matching across hierarchical structures
- **Comparative queries** (20): Queries using node comparisons (<, >, . .)
- **Complex queries** (20): Multi-constraint queries combining relations and features

The runner executes each query 50 times, with 5 runs × 10, excluding warmup iterations. Results are reported using mean latency, standard deviation, and 95% confidence intervals. The results are shown in Table 5 and Figure 4.

Table 5: Mean query latency (ms) by category. Positive speedup indicates CF is faster; negative indicates CF is slower.

Category	Queries	TF (ms)	CF (ms)	Speedup
Feature	30	221.5	164.0	+26%
Structural	30	179.2	190.5	−6%
Comparative	20	297.3	286.1	+4%
Complex	20	534.5	554.9	−4%
Overall	100	286.0	278.2	+3%

Across feature-based lookups, CF attains a 26% speed-up, owing to its vectorized lookup for simultaneous nodes. On structural / complex queries, CF lags TF by 4–6% due to the memory mapping page-fault penalty. Averaging across all query types gives CF a negligible 3% gain over TF. Given the tradeoffs documented above, this is a positive result for CF.

[7] The anomalous dip at step ten is likely attributable to Python's garbage collection program getting triggered. Two of five runs showed ∼1,750 MB drops at this step, while others remained consistent on the curve.

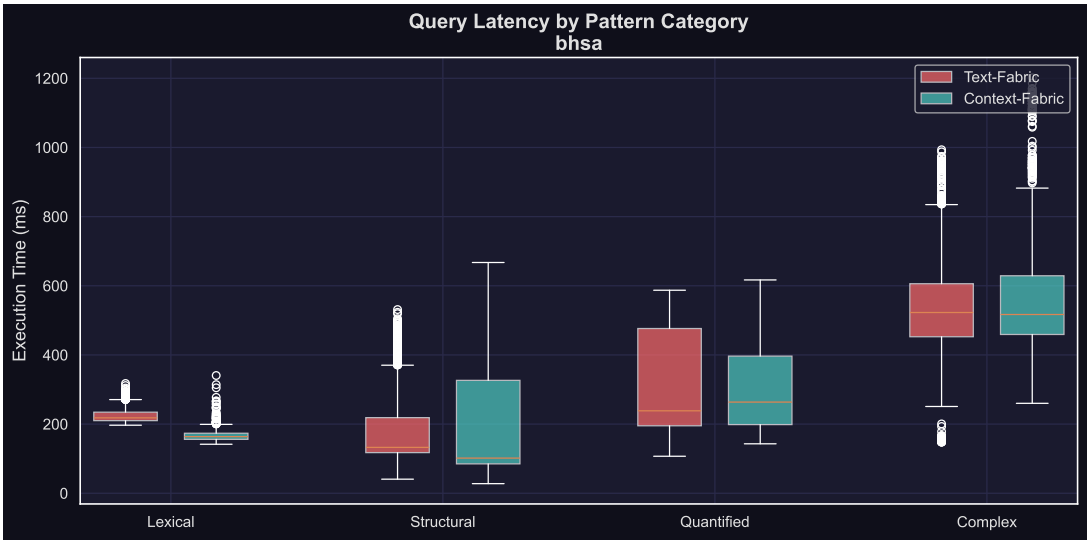


Figure 4: Query latency distribution by category. Boxes show interquartile range; whiskers extend to 1.5× IQR.

4 Model Context Protocol (MCP)

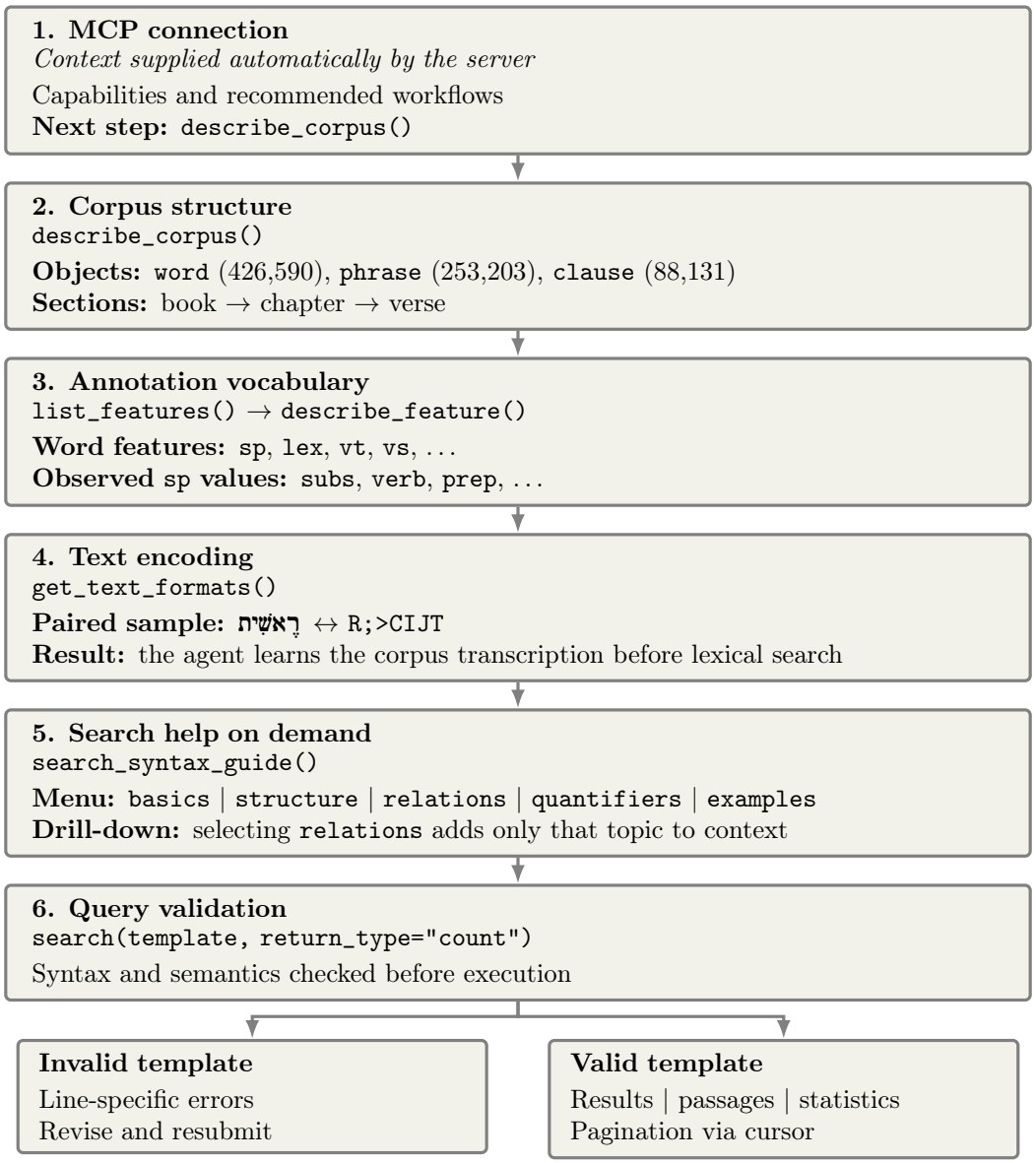
To enable AI agents to interact with CF, it needs to provide functions they can call for discovering and searching corpora. A common agentic framework is Model Context Protocol (MCP), which exposes self-describing endpoints to an agent's context window, which it can invoke using structured arguments [30]. Typically, these functions would be optimized for "progressive discovery", allowing the agent to route to the most specific tool it needs for its task.

Context-Fabric ships with a dedicated MCP server which automatically describes corpora and the available features. It is multi-corpora by design. Every agent begins with CF's server description "screen", which shows the text found in Listing 8. This view orients the agent and provides it with a suitable entry point. The typical tool-call flow for an agent with CF's MCP server is illustrated in Figure 5.

```
Context-Fabric exposes corpora for structured analysis. Corpora contain hierarchical
objects (e.g., words, sentences, documents) with arbitrary annotations. A corpus can
be annotated for almost anything: linguistics, music, manuscripts, DNA sequences,
legal documents, etc.

## Capabilities
- Search for patterns using structural templates
- Filter by any annotated feature on any object type
- Analyze feature distributions across search results
- Retrieve passages by section reference
- Explore hierarchical relationships between objects

## Workflows

### Explore corpus structure
Understand what's in the corpus before searching:
1. describe_corpus() - Get node types and section structure
2. list_features(node_types=[...]) - See features for a node type
3. describe_feature('feature_name') - Get sample values for a feature

### Understand text encoding
Learn how text is encoded before lexical searches
(critical for lex=... or surface text queries):
1. get_text_formats() - See original script and transliteration samples
2. Use samples to construct accurate search patterns

### Search for patterns
Find patterns matching structural templates:
1. search_syntax_guide() - Learn search template syntax
2. search(template, return_type='count') - Check result count
3. search(template) - Get actual results
   (validation errors returned automatically)

### Analyze distributions
Understand feature patterns across results:
- search(template, return_type='statistics', aggregate_features=[...])

### Read passages
Retrieve text by section reference:
- get_passages([[section_ref], ...])

## Tips
- Start with describe_corpus() to understand the structure
- Use list_features(node_types=[...]) to find relevant features
- Call get_text_formats() before lexical/surface text searches
- Use search(..., return_type='count') before fetching full results
- Invalid templates return detailed error messages automatically

## Next Step
Call describe_corpus() to see node types and section structure.
```

Listing 8: Full connection-time instructions supplied to the agent

Figure 5: The agent's progressive view through the MCP server.



4.1 Automatic Corpus Description

One challenge with designing the CF MCP server is that TF corpora documentation is sporadic. Ideally, the server should work well regardless of documentation coverage. Furthermore, it should work out-of-the-box without extensive configuration. To do this, CF needs to dynamically conduct a graph analysis on every corpus it loads. This is done by analyzing the obligatory metadata such as `slotType`, `otype` (object type of nodes), as well as making statistical counts of node and edge features. Additionally, `.tf` files contain optional metadata which provide human-readable descriptions for features. These are used if available.

Four critical tools expose the relevant information to the agent. `describe_corpus` provides a breakdown of the node types and sectional hierarchy. `list_features` returns a feature catalogue with optional filters for node `otype`. `describe_feature` provides illustrative values for a given feature, sorted by frequency, along with any description metadata. `get_text_formats`. The agent's interactions with these three is illustrated in Listing 9.

```
describe_corpus() -> {
  "node_types": [
```

```
    {"type": "clause", "count": 88131, "is_slot_type": false},
    {"type": "phrase", "count": 253203, "is_slot_type": false},
    {"type": "word", "count": 426590, "is_slot_type": true}
  ],
  "sections": {"levels": ["book", "chapter", "verse"]}
}

list_features(node_types=["word"]) -> {  # selected entries
  "features": [
    {"name": "lex", "kind": "node", "value_type": "str"},
    {"name": "sp", "kind": "node", "value_type": "str"},
    {"name": "vs", "kind": "node", "value_type": "str"},
    {"name": "vt", "kind": "node", "value_type": "str"}
  ],
  "node_types": ["word"]
}

describe_feature("sp") -> {
  "node_types": ["lex", "word"],
  "unique_values": 14,
  "sample_values": [
    {"value": "subs", "count": 125583},
    {"value": "verb", "count": 75451},
    {"value": "prep", "count": 73298}
  ]
}
```

Listing 9: Agent view of corpus description tools

The fourth tool is the `get_text_formats` tool. It addresses a particularly tricky problem: how to enable an agent to understand a variety of transcription formats without a transcription table? The BHSA, for example, has a detailed table available online, but it is not stored within the `.tf` data. However, the `.tf` format *does* reserve special metadata for identifying transcription features, which powers TF's `Text` API. In this format, the tag `orig` ("original") designates one feature as the reference script, while others are described as `trans` ("transcription"). Using the `orig`, the CF MCP loader scans all slots in the corpus and saves each one containing a novel UTF-8 character, until it exhausts all novel characters. This typically results in a few dozen sampled slots, with their original text. Then, for any given transcription format, the `get_text_formats` tool can show an exhaustive side-by-side, with all original characters on one side, and all transcriptions on the other. This approach cleanly handles cases where the transcription format does not match one-for-one the original text. Agents are then able to reason and infer over this sample when crafting their search strings. A sampling of the agent view for this tool is shown in Listing 10.

```
get_text_formats() -> {
  "formats": [{
    "name": "lex-full",
    "original_script": "{g_lex_utf8} ",
    "transliteration": "{g_lex} ",
    "samples": [  # selected subset; 4 of 28 shown
      {"original": "בְּ", "transliterated": "B.:-"},
      {"original": "רֵאשִׁית", "transliterated": "R;>CIJT"},
      {"original": "בָּרָא", "transliterated": "B.@R@>"},
      {"original": "אֱלֹה", "transliterated": ">:ELOH"}
    ],
    "unique_characters": 43,
    "total_samples": 28
  }]
}
```

Listing 10: Agent view of the `get_text_formats` output

4.2 Search Guide

The TF query language is how agents execute searches with CF's MCP server. In order to onboard agents to the query language, a set of tools provides the necessary syntax guides. `search_syntax_guide()` returns a brief summary along with a menu of options for further exploration: basics, structure, relations, quantifiers, and examples. The agent can then call `search_syntax_guide(section=...)` to retrieve one of these topics. Frontier model agents are surprisingly adept at mastering the query language. Syntax errors in the execution engine return an informative remediation message indicating the specific offending line. Agents can then resubmit the corrected query string. Results are returned as paginated views, which the agent can progressively scroll through. Samples of the search guide are shown in Listing 11.

```
search_syntax_guide() -> {
  "summary": "Templates: node_type feature=value. Indentation=containment.
              Relations: < > <: >: for ordering.",
  "sections": ["basics", "structure", "relations", "quantifiers", "examples"],
  "hint": "Call with section='relations' to get detailed info on a specific section"
}

search_syntax_guide(section="relations") ->

## Relations

### Default Relations
- Indented items are contained by their parent (‘:‘ relation)
- Items at same level follow each other in order

### Explicit Relations
clause
  word sp=verb
  < word sp=noun              # noun comes BEFORE verb
  > word sp=adj               # adjective comes AFTER verb
  <: word sp=prep             # preposition immediately before verb
  :> word sp=adv              # adverb immediately after verb

### Relation Operators
- ‘<‘  - comes before (canonical node ordering)
- ‘>‘  - comes after (canonical node ordering)
- ‘<:‘ - immediately before (adjacent)
- ‘:>‘ - immediately after (adjacent)
- ‘<<‘ - completely before (slot ordering)
- ‘>>‘ - completely after (slot ordering)
- ‘[[‘ - left embeds right
- ‘]]‘ - left embedded in right
- ‘=:‘ - start at same slot
- ‘:=‘ - end at same slot
- ‘::‘ - start and end at same slot (co-extensive)
- ‘==‘ - occupy same slots
```

Listing 11: Agent view of the progressive search guide

5 Conclusion

Context-Fabric is open source and freely available on GitHub or PyPI.[8] It can be installed with the following command:

```
pip install context-fabric[mcp]
```

Future optimizations of Context-Fabric are already in the works. An experimental re-write in Rust (with Python bindings) is showing promising, further performance gains.[9] There is also work underway to improve the agentic ergonomics of the MCP server. Interested readers may

[8] See `https://github.com/Context-Fabric/context-fabric` and `https://pypi.org/project/context-fabric/`.

[9] See `https://github.com/Context-Fabric/context-fabric/tree/cf-rust`.

follow the CF GitHub organization, at https://github.com/Context-Fabric/, or visit the CF website at https://context-fabric.ai.

It is my hope that CF can help further Roorda's vision, of a Biblical Studies and Humanities (more broadly) that is transformed by statistical science. If LLMs can teach us anything, it is that the patterns of language are, surprisingly, recoverable and objective—at least in their public, conventionalized setting. That is to say nothing of human beings' penchant for *unpredictability*, something I personally doubt will ever be captured by a statistical model. Therein lies the optimism for an AI-saturated future. Not one where humans become irrelevant, but where their existing creativity is amplified and extended. The skill premium will undoubtedly shift towards curiosity, challenging in a time where machines can do all the work for you. Those who continue to exercise this muscle will reap the benefits.

References

[1] Oosting, R. (2016). Computer-assisted analysis of Old Testament texts: The contribution of the WIVU to Old Testament scholarship. In K. Spronk (Ed.), *The Present State of Old Testament Studies in the Low Countries* (pp. 192–209). Brill. https://doi.org/10.1163/9789004326255

[2] Roorda, D., Kalkman, G., Naaijer, M., & van Cranenburgh, A. (2014). LAF-Fabric: A data analysis tool for Linguistic Annotation Framework with an application to the Hebrew Bible. *Computational Linguistics in the Netherlands Journal*, 4, 105–120. https://clinjournal.org/clinj/article/view/44

[3] Ide, N., & Suderman, K. (2014). The Linguistic Annotation Framework: A standard for annotation interchange and merging. *Language Resources and Evaluation*, 48, 395–418. https://doi.org/10.1007/s10579-014-9268-1

[4] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[5] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

[6] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[7] Naaijer, M., & Roorda, D. (2016). Parallel texts in the Hebrew Bible: New methods and visualizations. *arXiv preprint arXiv:1603.01541*. https://arxiv.org/abs/1603.01541

[8] Kalkman, G. J. (2015). *Verbal forms in Biblical Hebrew poetry: Poetic freedom or linguistic system?* [PhD thesis, Vrije Universiteit Amsterdam]. https://research.vu.nl/en/publications/verbal-forms-in-biblical-hebrew-poetry-poetic-freedom-or-linguist

[9] Kingham, C. A. (2016). *Clause syntax in the Song of Songs: A preliminary study* [Master of Arts thesis, Southeastern Baptist Theological Seminary]. GitHub: https://github.com/codykingham/SongofSongs-Syntax

[10] Erwich, C. M. (2021). *Who is who in the Psalms? Coreference resolution as exegetical tool for participant analysis in Biblical texts* [PhD thesis, Vrije Universiteit Amsterdam]. https://research.vu.nl/en/publications/who-is-who-in-the-psalms-coreference-resolution-as-exegetical-too

[11] Højgaard, C. C. (2021). *Roles and relations in Biblical law: A study of participant tracking, semantic roles, and social networks in Leviticus 17–26* [PhD thesis, Vrije Universiteit Amsterdam]. https://research.vu.nl/en/publications/roles-and-relations-in-biblical-law-a-study-of-participant-tracki/

[12] van Peursen, W. (2019). A computational approach to syntactic diversity in the Hebrew Bible. *Journal of Biblical Text Research*, 44, 237–253. https://doi.org/10.28977/jbtr.2019.4.44.237

[13] Roorda, D. (2016). *Text-Fabric* [Computer software]. GitHub. https://pure.knaw.nl/portal/en/publications/text-fabric/

[14] OpenAI. (2022, November 30). Try talking with ChatGPT, our new AI system which is optimized for dialogue [Post]. X. https://x.com/OpenAI/status/1598014522098208769

[15] PyTorch Contributors. (n.d.). Tips for loading an `nn.Module` from a checkpoint. *PyTorch Tutorials*. https://docs.pytorch.org/tutorials/recipes/recipes/module_load_state_dict_tips.html

[16] Google AI Edge. (n.d.). Build LiteRT with CMake. *LiteRT Documentation*. https://developers.google.com/edge/litert/build/cmake

[17] NumPy Developers. (n.d.). `numpy.memmap`. *NumPy Documentation*. https://numpy.org/doc/stable/reference/generated/numpy.memmap.html

[18] van Peursen, W. Th., Sikkel, C., & Roorda, D. (2015). Hebrew Text Database BHSA. DANS. https://doi.org/10.17026/dans-z6y-skyh. Data curated by the Eep Talstra Centre for Bible and Computer, Vrije Universiteit Amsterdam.

[19] Center for Biblical Languages and Computing. (2024). Septuagint (Rahlfs' LXX Edition 1935) in Text-Fabric. Based on Eliran Wong's RLXX1935 dataset. GitHub: https://github.com/CenterBLC/LXX

[20] Center for Biblical Languages and Computing. (2024). Nestle 1904 Greek New Testament in Text-Fabric. GitHub: https://github.com/CenterBLC/N1904

[21] Jacobs, J., Naaijer, M., & Roorda, D. (2017). Dead Sea Scrolls in Text-Fabric. Data provided by Martin Abegg. Developed as part of the CACCHT project. GitHub: https://github.com/ETCBC/dss

[22] van Peursen, W. Th., Veldman, G. J., Sikkel, C., Vlaardingerbroek, H., & Roorda, D. (2018). ETCBC/peshitta: Syriac Old Testament (Peshitta) in Text-Fabric. Zenodo. https://doi.org/10.5281/zenodo.1464757

[23] Vlaardingerbroek, H. & Roorda, D. (2017). Syriac New Testament in Text-Fabric. Source data from SEDRA database by George A. Kiraz and James W. Bennett. GitHub: https://github.com/ETCBC/syrnt

[24] Naaijer, M., Højgaard, C. C., Schorch, S., & Ehrensvärd, M. (2020). Samaritan Pentateuch in Text-Fabric. Developed as part of the CACCHT project. GitHub: https://github.com/DT-UCPH/sp

[25] Kingham, C. (2018). Tischendorf 8th Edition Greek New Testament in Text-Fabric. Based on Ulrik Sandborg-Petersen's corpus. GitHub: https://github.com/codykingham/tischendorf_tf

[26] van Lit, C. & Roorda, D. (2017). Quranic Arabic Corpus in Text-Fabric. Source data from the Quranic Arabic Corpus and Tanzil project. GitHub: https://github.com/q-ran/quran

[27] Højgaard, C. C., Naaijer, M., Ehrensvärd, M., et al. (2020). Copenhagen Ugaritic Corpus in Text-Fabric. Developed as part of the CACCHT project. GitHub: https://github.com/DT-UCPH/cuc

[28] Beyer, D., Löwe, S., & Wendler, P. (2019). Reliable benchmarking: Requirements and solutions. *International Journal on Software Tools and Technology Transfer*, 21, 1–29. https://doi.org/10.1007/s10009-017-0469-y

[29] Psutil Contributors. (n.d.). Process memory information. *psutil Documentation*. https://psutil.readthedocs.io/stable/index.html#psutil.Process.memory_info

[30] Model Context Protocol Contributors. (2025). *Model Context Protocol Specification*, revision 2025-11-25. https://modelcontextprotocol.io/specification/2025-11-25